

Министерство образования Республики Беларусь  
Учреждение образования  
«Брестский Государственный технический университет»  
Кафедра ИИТ

**Лабораторная работа №4**

**По дисциплине:** «Аппаратно-программное обеспечение ЭВМ и сетей»  
**Тема:** «Архитектура и программирование сопроцессора. Использование  
целочисленных команд. Использование вещественных команд»

**Выполнила:**  
Студент 4 курса  
Группы АС-66  
Езепчук А.С.  
**Проверила:**  
Кулик А.Д.

**Цель работы:** изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных команд

### Вариант 6

**Постановка задачи раздела 1.** Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и целочисленные команды согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям.

### Задание раздела 1:

$$\text{Вариант 6}$$
$$y = \begin{cases} 5x + 2xy + 1, & x + y > 9; \\ \frac{2xy + 3}{y^2 + 4}, & x + y < -5; \\ 3x^2 - 2y^2 + 1, & -5 \leq x + y \leq 9. \end{cases}$$

(10 шакалов из 10)

### Код программы

```
.386
.model flat, stdcall
option casemap:none

includelib ucrt.lib
includelib legacy_stdio_definitions.lib
includelib kernel32.lib

printf PROTO C :PTR BYTE, :VARARG
scanf PROTO C :PTR BYTE, :VARARG
ExitProcess PROTO STDCALL :DWORD

.data
prompt db "Enter x and y: ", 0
fmt_in db "%d %d", 0
fmt_out db "y = %.6f", 13, 10, 0

five dd 5
four dd 4
three dd 3
two dd 2
one dd 1

.code

CalcHigh PROC x:SDWORD, y:SDWORD
    LOCAL a:REAL8
    LOCAL b:REAL8

    fild x
    fimul five
    fstp a

    fild x
    fild y
```

```

    fmul
    fimul two
    fstp b

    fld a
    fadd b
    fiadd one

    ret
CalcHigh ENDP

CalcLow PROC x:SDWORD, y:SDWORD
    LOCAL a:REAL8
    LOCAL b:REAL8

    fld x
    fld y
    fmul
    fimul two
    fiadd three
    fstp a

    fld y
    fmul st(0), st(0)
    fiadd four
    fstp b

    fld a
    fdiv b

    ret
CalcLow ENDP

CalcMiddle PROC x:SDWORD, y:SDWORD
    LOCAL a:REAL8
    LOCAL b:REAL8

    fld x
    fmul st(0), st(0)
    fimul three
    fstp a

    fld y
    fmul st(0), st(0)
    fimul two
    fstp b

    fld a
    fsub b
    fiadd one

    ret
CalcMiddle ENDP

CalcFunc PROC x:SDWORD, y:SDWORD
    mov eax, x
    add eax, y

    cmp eax, 9
    jg high_branch

    cmp eax, -5
    jl low_branch

    jmp middle_branch

high_branch:
    invoke CalcHigh, x, y
    ret

low_branch:
    invoke CalcLow, x, y
    ret

middle_branch:

```

```

        invoke CalcMiddle, x, y
        ret

CalcFunc ENDP

main PROC
    LOCAL x:SDWORD
    LOCAL y:SDWORD

    invoke printf, ADDR prompt
    invoke scanf, ADDR fmt_in, ADDR x, ADDR y

    invoke CalcFunc, x, y

    sub esp, 8
    fstp qword ptr [esp]

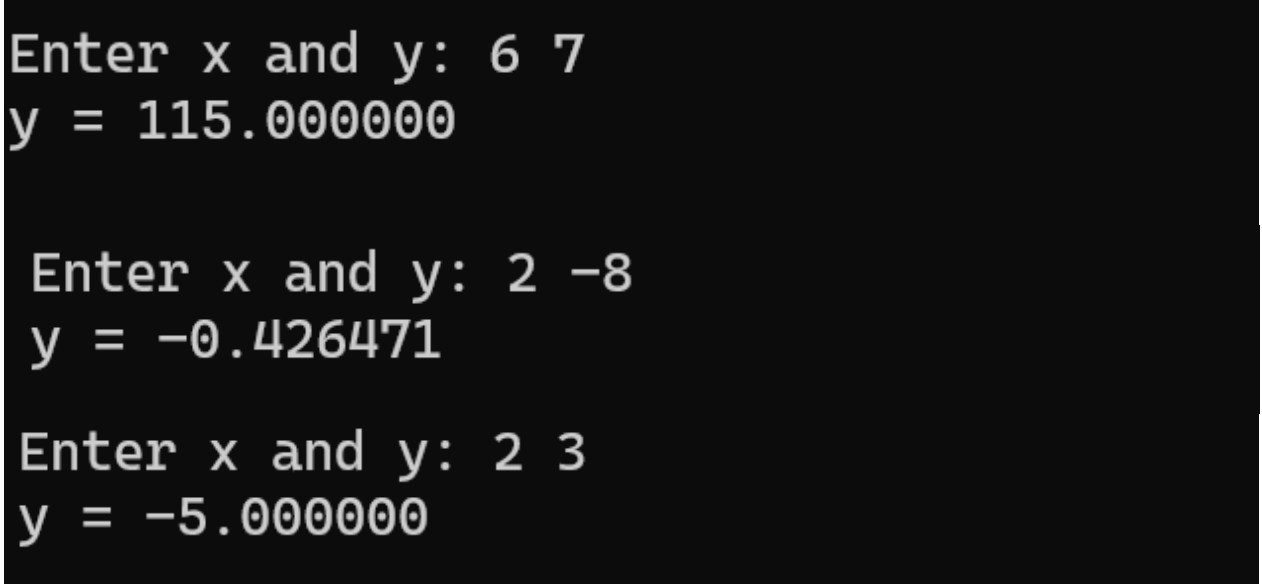
    push offset fmt_out
    call printf
    add esp, 12

    invoke ExitProcess, 0
main ENDP

END main

```

### Результат работы программы:



```

Enter x and y: 6 7
y = 115.000000

Enter x and y: 2 -8
y = -0.426471

Enter x and y: 2 3
y = -5.000000

```

**Постановка задачи раздела 2.** Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и вещественные команды согласно варианту, выданному преподавателем (таблица 2.2). Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям

**Задание раздела 2:** тоже самое

### Код программы

```

.386
.model flat, stdcall
option casemap:none

includelib ucrt.lib
includelib legacy_stdio_definitions.lib
includelib kernel32.lib

```

```
printf PROTO C :PTR BYTE, :VARARG
scanf PROTO C :PTR BYTE, :VARARG
ExitProcess PROTO STDCALL :DWORD
```

```
.data
prompt db "Enter real x and y (e.g. 2.5 -3.75): ", 0
fmt_in db "%lf %lf", 0
fmt_out db "Real result: y = %.6f", 13, 10, 0
```

```
two REAL8 2.0
three REAL8 3.0
four REAL8 4.0
five REAL8 5.0
one REAL8 1.0
nine REAL8 9.0
minus5 REAL8 -5.0
```

```
.code
```

```
BranchHigh PROC px:DWORD, py:DWORD
```

```
    mov eax, px
    fld qword ptr [eax]
    fmul qword ptr [five]
```

```
    mov eax, px
    fld qword ptr [eax]
```

```
    mov eax, py
    fmul qword ptr [eax]
    fmul qword ptr [two]
```

```
    faddp st(1), st(0)
    fadd qword ptr [one]
```

```
    ret
```

```
BranchHigh ENDP
```

```
BranchLow PROC px:DWORD, py:DWORD
```

```
    mov eax, px
    fld qword ptr [eax]
```

```
    mov eax, py
    fmul qword ptr [eax]
    fmul qword ptr [two]
    fadd qword ptr [three]
```

```
    mov eax, py
    fld qword ptr [eax]
    fmul st(0), st(0)
    fadd qword ptr [four]
```

```
    fdivp st(1), st(0)
```

```
    ret
```

```
BranchLow ENDP
```

```
BranchMiddle PROC px:DWORD, py:DWORD
```

```
    mov eax, px
    fld qword ptr [eax]
    fmul st(0), st(0)
    fmul qword ptr [three]
```

```
    mov eax, py
    fld qword ptr [eax]
    fmul st(0), st(0)
    fmul qword ptr [two]
```

```
fsubp st(1), st(0)
fadd qword ptr [one]
```

```
ret
```

```
BranchMiddle ENDP
```

```
CalcFunc PROC px:DWORD, py:DWORD
```

```
LOCAL sum:REAL8
```

```
mov eax, px
fld qword ptr [eax]
```

```
mov eax, py
fadd qword ptr [eax]
```

```
fstp qword ptr [sum]
```

```
fld qword ptr [sum]
fcom qword ptr [nine]
fstsw ax
fstp st(0)
```

```
test ax, 0100h
jnz check_low
```

```
test ax, 4000h
jnz check_low
```

```
invoke BranchHigh, px, py
ret
```

```
check_low:
```

```
fld qword ptr [sum]
fcom qword ptr [minus5]
fstsw ax
fstp st(0)
```

```
test ax, 0100h
jnz use_low
```

```
invoke BranchMiddle, px, py
ret
```

```
use_low:
```

```
invoke BranchLow, px, py
ret
```

```
CalcFunc ENDP
```

```
main PROC
```

```
LOCAL x:REAL8
LOCAL y:REAL8
LOCAL result:REAL8
```

```
finit
```

```
invoke printf, ADDR prompt
invoke scanf, ADDR fmt_in, ADDR x, ADDR y
```

```
invoke CalcFunc, ADDR x, ADDR y
```

```
fstp qword ptr [result]
```

```
push dword ptr [result+4]
push dword ptr [result]
push OFFSET fmt_out
```

```
call printf
add esp, 12

invoke ExitProcess, 0

main ENDP

END main
```

### Результат работы программы:

```
Enter real x and y (e.g. 2.5 -3.75): 3.6 5.5
Real result: y = 58.600000

Enter real x and y (e.g. 2.5 -3.75): -4.2 -3.5
Real result: y = 1.993846

Enter real x and y (e.g. 2.5 -3.75): 2.3 3.2
Real result: y = -3.610000
```

**Вывод:** в ходе лабораторной работы была изучена архитектура и программная модель математического сопроцессора, а также основные принципы работы с его регистрами и командами. На практике были освоены загрузка и обработка вещественных чисел, выполнение арифметических операций и организация вычислений с использованием стека сопроцессора.